WORKSHEET: LOGISTIC REGRESSION

Topic: Machine Learning / Classification | Duration: 45 Minutes

Name: _____ Roll Number: _____ Date: _____

What is Logistic Regression?

Do not let the name fool you!

❌ **It is NOT "Regression":** Regression is for predicting numbers (like temperature).

✅ **It IS "Classification":** It is used to separate data into categories (Yes/No, Spam/Not Spam).

Key Idea: It predicts the **Probability (0% to 100%)** that an event will happen.

> *"Classification is the art of drawing a line in the sand. You are either on this side, or that side."*
>
> ***— The Bouncer Analogy***

The Concept:

Imagine you are a bouncer at a club. You do not care exactly how old someone is or how much money they have. You only have one question to answer: **"Are they allowed inside? Yes or No?"**

That is Logistic Regression. Before we can build this "Bouncer" (our model), we must first prepare our environment. Just like a chef prepares ingredients before cooking, we must load and inspect our data before modeling.

Phase 1: Data Preprocessing

Before any learning can happen, we must load and understand our data. Think of this as opening the textbook before studying.

The Dataset: We are using a dataset of student attendance and their pass/fail status.

attendance_percent	passed
45	0
50	0
52	0
...	...
65	1
...	...
90	1

Look at the data table.

0 = Failed

1 = Passed

Question: As attendance increases, what happens to the 'passed' column?

- ☐ It tends to stay 0
- ☐ It switches to 1

Let's Load Our Data

We use Python libraries to handle data. **Type the following code exactly** to import libraries and load the CSV file.

```python
import numpy as np
import pandas as pd

# Load the dataset
df = pd.read_csv('attendance.csv')
```

Knowledge Check:

1. `pd` is an alias for the library: _____

2. The function `read_csv` is used to:

   - ☐ Draw a graph
   - ☐ Load data from a file

Step 2: Start Inspecting the Dataset

After loading, we need to peek inside. We use `.head()` to see the first 5 rows.

```python
# View the top 5 rows
print(df.head())
```

Your Turn:

If `head()` shows the *top* rows, what command do you think shows the *bottom* rows?

`print(df.` _____ `())`

Phase 2: The Logic (The S-Curve)

The Problem: Why can't we just use a straight line (Linear Regression)?

A straight line goes to infinity (∞). But probability must always be between **0 and 1** (0% and 100%). We need a way to "squash" our line so it never goes above 1 or below 0.

The Solution: The Sigmoid Function (S-Curve)

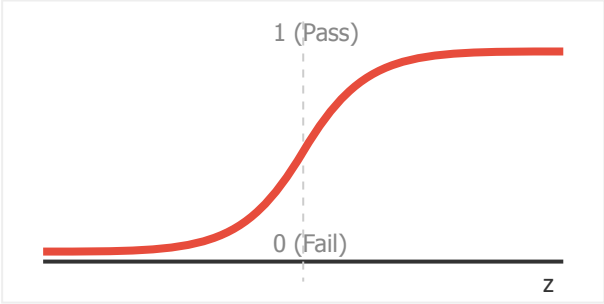
It takes any number (z) and squashes it between 0 and 1.

$$P = \frac{1}{1+e^{-z}}$$

But what is z?
It is just our old friend, the linear equation (Line):

$$z = mx + c$$

(Input $\times$ Weight + Bias)



Visual: The S-Curve squashing values between 0 and 1

- If z is very high, P becomes 1 (Pass).
- If z is very low, P becomes 0 (Fail).

Phase 3: Building the Model (Scikit-Learn)

Now we will build the "Bouncer" using Python's **scikit-learn** library. Follow the steps and fill in the code.

Step 1: Separate Input and Output

We need to tell the computer what are the "Ingredients" (X) and what is the "Target" (y).

```python
# X = Features (Attendance) | y = Target (Passed)
X = df[['attendance_percent']] # Note output: 2D Array [[ ]]
y = df['passed']
```

Step 2: Import & Initialize

We import the empty brain of our model.

```python
from sklearn.linear_model import LogisticRegression

# Create the model (The Bouncer)
model = LogisticRegression()
```

Step 3: Train the Model (Fit)

This is where the magic happens. We teach the Bouncer using our data.

```python
# Teach the model
model._____(X, y)
```

Hint: The command to train a model is named after "fitting" a line to data.

Step 4: Make a Prediction

Now the Bouncer is ready. Let's ask: *"Will a student with 85% attendance pass?"*

```python
# Predict for 85% attendance
new_student = [[85]]
```

```
prediction = model._________(new_student)
print(prediction)
```

Output Interpretation:

*If the output is **[1]**, the student Passes.*

*If the output is **[0]**, the student Fails.*